The first thing this code does is find the train lines that will take the user from the departure station to the destination. This is the domain of the timetable service, so you ask it to provide you with a list of lines going from one station to another ❶. In this context, a *line* represents the path that trains take from one terminus to another, in a given direction. Remember, the timetable service doesn't have any methods yet, so you've just discovered something that you need the timetable service to do.

Once you have a list of the lines, you can find the times of the trains arriving at the departure station on any line. There are a few ways you could do this, but you'll delegate it to another method ❷ and move on with the main business logic.

The last thing this method needs to do is to find the arrival times after the requested starting time ❸ and return the next three departure times ❹.

You can see the complete source code on GitHub,[14] but let's focus on the `getArrivalTimesOnLines()` method, shown here:

```
private List<LocalTime> getArrivalTimesOnLines(List<Line> lines,
                                               String station) {
    TreeSet<LocalTime> allArrivalTimes = Sets.newTreeSet();
    for(Line line : lines) {                                          ❶ Collect
        allArrivalTimes.addAll(                                         arrival
            timetableService.findArrivalTimes(line,station));           times
    }
    return new ArrayList<LocalTime>(allArrivalTimes);
}
```

This method is interesting because it points out something else you need the timetable service to do. The logic isn't complicated—it just collects the arrival times of all the lines going through a given station ❶. A `Line` is a simple domain object with a name, a departure station, and a list of stations on that line.[15]

But for the `getArrivalTimesOnLines()` method to work, you need to know the times that trains on a particular line are due to arrive at the departure station, and you need to get this information from the timetable service.

This means you need a `TimetableService` object that does the following:

- Finds the lines through any two stations.
- Finds the arrival times for a given line at a particular station.

You can formalize what you need in the *Given* step of your Spock specification by creating a "pretend" timetable service that behaves exactly as you'd like it to, and ensuring that your itinerary service processes the data it gets from the timetable service correctly. You could do this with the following code:

```
TimetableService timetableService = Mock(TimetableService)   <- Create a "pretend" timetable service.

def "should calculate the correct arrival time"() {
  given:
```

[14] The GitHub page is at https://github.com/bdd-in-action/chapter-2.

[15] For simplicity, the full implementation of the `Line` class is provided in the sample code.